

ControlPlane.ai — Prototype README

Real-time, configurable AI oversight for the enterprise

Accenture Innovation Challenge 2026 — Round 2 | Team: Innovators | Jay Chandwani

This document accompanies the public GitHub repository containing a working prototype of the core ControlPlane.ai mechanism described in the Business Proposal: a checker that scores AI responses in real time and applies a use-case-specific, configurable policy — allow, edit, block, or escalate — rather than a single fixed rule set.

1. What This Prototype Demonstrates

- Rule-based PII / entity detection — deterministic, fast, sidesteps the ground-truth problem entirely.
- AI-as-judge scoring — a relative plausibility/consistency check (real LLM call if an API key is configured, otherwise a transparent simulated fallback so the repo runs instantly for any reviewer).
- Policy-driven tiered decisions — the same detection logic, evaluated under three different persona configs, produces different outcomes. This is the central claim of the proposal: configurability lives in the policy, not in hard-coded branching logic.
- An audit trail — every decision is logged with a hash of the input, the scores that produced it, and the exact policy version applied.

2. Architecture

```
User Request -> AI Model -> [ControlPlane Checker] -> Response to User

|
|-- Fast-path (inline, blocking)
|   - PII / entity regex
|   - Policy config lookup
|
|-- Slow-path (async, non-blocking)
|   - AI-as-judge scoring
|   - Logged for escalation if needed
```

Full architecture and design rationale is in the Business Proposal (ControlPlane_Business_Proposal.pdf), Sections 4.4-4.5.

3. Project Structure

```
controlplane/
|-- controlplane/
|   |-- types.py           # shared dataclasses
|   |-- detectors/
|       |-- pii_detector.py # regex + name-heuristic PII detection
|       |-- judge_detector.py # AI-as-judge (LLM call or simulated fallback)
|   |-- policy/
|       |-- loader.py      # loads per-use-case YAML policy config
|   |-- engine/
|       |-- decision.py    # allow / edit / block / escalate logic
|   |-- audit/
|       |-- logger.py      # append-only JSONL audit trail
|-- configs/              # one policy YAML per persona / use case
|-- sample_data/
```

```
| `-- sample_responses.json # simulated AI responses used by the demo
|-- demo.py                # end-to-end CLI demo (run this)
`-- requirements.txt
```

4. Dependencies

- Python 3.10+
- pyyaml — policy config parsing
- rich — demo console output
- requests — used for the optional real-LLM judge call via Groq
- anthropic — optional alternative to Groq for the real-LLM judge path (paid; commented out in requirements.txt by default)

Install with:

```
pip install -r requirements.txt
```

5. Execution Instructions

5.1 Run the demo

```
python demo.py
```

This runs the full sample set (9 responses across 3 use cases) through the pipeline, prints a decision report, then runs a configurability demo: the same response evaluated under two different policies, to show the tier changing while the judge score stays identical.

5.2 Filter to one use case

```
python demo.py --use-case credit_decision_support
```

5.3 Optional: use a real LLM judge

By default, the judge detector runs in simulated mode - a transparent, deterministic heuristic scorer, clearly labeled as such in every output - so the prototype is reviewable without requiring your own API key. To use a real LLM as the judge instead, there are two options.

Option A - Groq (recommended, free). Groq offers a free API key with no payment required - sign up at console.groq.com/keys.

```
export GROQ_API_KEY=your_key_here
python demo.py
```

Option B - Anthropic (paid alternative). If you'd rather use a Claude model, uncomment anthropic in requirements.txt, install it, then:

```
export ANTHROPIC_API_KEY=your_key_here
python demo.py
```

If both GROQ_API_KEY and ANTHROPIC_API_KEY are set, Groq is used first. No other code changes are needed for either option - the decision engine only ever consumes a JudgeResult, regardless of which mode produced it.

6. Sample Output

ID	Use Case	Judge	Mode	Tier	Reason
cs-001	customer_support_chatbot	77	simulated	ALLOW	score at/above allow threshold (65)
cs-002	customer_support_chatbot	76	simulated	BLOCK	PII detected: EMAIL, PHONE
cs-003	customer_support_chatbot	37	simulated	ESCALATE	score below escalate threshold (40)
cd-003	credit_decision_support	62	simulated	BLOCK	PII detected: GOV_ID

...followed by the configurability demo, where one input is scored 82 under two different policies and produces ALLOW under the customer-support config but EDIT under the stricter credit-decision config. Every decision is also appended to audit_log.jsonl in the repo root.

7. Known Limitations

This prototype demonstrates the mechanism, not a production-grade system. Specific simplifications, all discussed as extensions in the Business Proposal:

- PII detection is regex + a name heuristic, not a trained NER model - a production deployment would use something like Microsoft Presidio or spaCy.
- The simulated judge mode is a heuristic stand-in, not a real hallucination detector - it exists so the pipeline is runnable without an API key. The real LLM-judge path is fully wired and works if a key is provided.
- No self-consistency sampling (asking the same prompt multiple times and comparing answers) - noted in the proposal as a future extension beyond this prototype's scope.
- No real async/queue infrastructure - the async vs. sync scoring mode is represented in the policy config and decision metadata, but the demo runs synchronously for simplicity; a production system would use a message queue for the async path.

8. Business Proposal

See ControlPlane_Business_Proposal.pdf for the full solution design, generalization argument, business case, phased rollout roadmap, and risk analysis this prototype implements a slice of.